FileFlow Studio

User Manual and Reference Guide

FileFlow Studio — Motor Visual de Automatización DAG en .NET 9

📖 User Manual and Node Reference Guide

FileFlow Studio v2.0

Node-Based File Automation and Batch Processing Platform built with .NET 9 and C# 13

📑 Table of Contents

---

1. Introduction and Design Philosophy

FileFlow Studio is a visual batch file automation platform designed to build interactive file processing pipelines, inspired by modern node-based tools such as *n8n*, *ComfyUI*, and *Node-RED*.

Key Features:

- **Plugin-Based Modularity:** Every functional area lives in an isolated, zero-touch plugin (`FileFlow.Plugin.*`).
- **Guaranteed Safety:** Non-destructive processing by default, virtual simulation (*Dry Run*), and safe deletion via Windows Recycle Bin.
- **Asynchronous .NET 9 Performance:** High-throughput reactive pipelines powered by `System.Threading.Channels` and in-memory SQLite telemetry (>82,000 logs/sec).
- **Nested Sub-Flows:** Encapsulate complex sub-graphs into reusable macro nodes with breadcrumb navigation (`Root › Macro A › Macro B`).
- **Dynamic Internationalization (i18n):** Instant on-the-fly language switching (English / Spanish) across the entire UI.

---

2. Fundamental Editor Concepts

Node Canvas (Nodify)

The canvas lets you drag and drop nodes from the left **Toolbox**. Each node features:

- **Input Ports:** Located on the left edge. Receive incoming files or trigger signals.
- **Output Ports:** Located on the right edge. Emit processed files or conditional branch items.
- **Status LED:** Visual glowing indicator reflecting the current state (`Idle` , `Running` , `Completed` , `Paused` , `Faulted`).
- **Breakpoint Toggle:** Click the top-left circle to pause execution when a file reaches that specific node.
- **Header Actions Bar (`CustomActions`):** Quick-access buttons (`🔑 Method Pipeline ...` , `➕ Add Variable` , `➕ Add Case` , `💻 Script Editor ...`) directly on the card.

The File Context (`FileItemContext`)

A lightweight, immutable/transmutable record traveling through DAG edges containing:

- `CurrentPath`: The current on-disk physical or virtual path of the file at this step.
- `OriginalPath`: The immutable entry path where the file was initially ingested.
- `FileSizeBytes`: Exact file size in bytes.
- `Metadata`: Key-value dictionary enriched by upstream nodes (`Exif:`, `Hash:`, `Cli:`, `Doc:`, etc.).
- `Tags`: A `HashSet` collection of labels and operational flags.
- `ExecutionLog`: Trace history of all transformations applied to this specific item.

Nested Sub-Flows and Multi-Level Macros (Breadcrumbs)

Sub-flows allow encapsulating complex graph sections into a single compound node:

1. Double-click the sub-flow node or click **"Open Sub-flow"**.

2. The canvas displays the inner graph and the top breadcrumb navigation bar: `Root Workflow › Sub-flow A › Sub-flow B`.

3. Clicking on any previous level automatically saves changes and returns to the parent canvas.

Real-Time Telemetry Badges

During pipeline execution, every connection wire updates dynamically with live item counts (e.g., `⚡ 1,250 items`), helping you pinpoint bottlenecks and observe data routing in real time.

---

3. Execution Modes and Data Safety

Normal Execution

Click the green **"▶ Run Workflow"** button in the top toolbar. The engine processes all files concurrently according to the configured degree of parallelism.

Virtual Simulation Mode ("Dry Run")

1. Check the **"Dry Run"** toggle or click **"Virtual Simulation"**.

2. The engine executes the complete graph resolving names, paths, hashes, and logic **without writing, moving, or deleting any physical files on disk**.

3. The bottom console outputs an exact list of all planned actions (`PlannedAction`).

Rollback System and Windows Recycle Bin

- **Safe Deletion:** All deletion operations use the native Windows Shell API (`SHFileOperation`), moving files to the Recycle Bin rather than permanently deleting them.
- **Undo / Rollback (Ctrl+Z):** If you wish to revert changes after running a flow, click the **"↩ Undo / Rollback"** button. The engine undoes renames and moves in LIFO order (last executed, first undone).

Interactive Debugging with Breakpoints

- Click **"🐞 Debug Workflow"**.

- Execution automatically pauses when an item reaches a node with an active breakpoint or when an exception occurs.
- Use **"Step Over (F10)"** to advance one node at a time and inspect metadata diffs in the **Node Inspector Panel**.

---

4. Token Engine and Dynamic Variables

Any text input, output directory, or file name template supports dynamic variables enclosed in `{ ... }`.

Syntax:

`{Domain:Key:Modifier}` or `{Variable}`

Available Token Catalog:

Token	Output Example	Description
:---	:---	:---
`{FileName}`	`document.pdf`	File name with extension
`{FileNameNoExt}`	`document`	File name without extension
`{Ext}` or `{Extension}`	`pdf`	File extension without leading dot
`{CurrentDir}`	`C:\MyFiles`	Current directory path
`{ParentDir}`	`MyFiles`	Name of the immediate parent folder
`{CreationDate:yyyyMMdd}`	`20260902`	Formatted creation date
`{ModifiedDate:yyyy-MM-dd}`	`2026-09-02`	Formatted last modification date
`{Now:yyyyMMdd_HHmmss}`	`20260902_143000`	Current system timestamp
`{FileSize:MB}` or `{SizeMB}`	`14.50`	Size in Megabytes
`{FileSize:KB}` or `{SizeKB}`	`14848.0`	Size in Kilobytes
`{Hash:SHA256}`	`e3b0c44298fc1c ...`	Full calculated SHA-256 hash
`{Hash:SHA256:8}`	`e3b0c442`	SHA-256 hash truncated to 8 characters
`{Exif:CameraModel}`	`Sony ILCE-7M4`	Camera model from EXIF metadata
`{Exif:Make}`	`SONY`	Camera manufacturer
`{Doc:PageCount}`	`18`	Total page count (PDF/Office docs)
`{Media:Duration}`	`00:45:12`	Video or audio duration
`{Env:USERPROFILE}`	`C:\Users\username`	OS environment variable
`{Index:D4}`	`0001`, `0002`	Batch sequential zero-padded counter

5. Exhaustive Node Catalog

Category 1: FileSystem (Disk I/O)

1. `FolderSourceNode` (Folder Source)

- **Purpose:** Scans a directory and emits discovered files into the pipeline.

- **Ports:**
- *Outputs:* `FileOut` (`FileItemContext`), `Completed` (End signal).
- **Parameters:**
- `SourceFolder` : Root folder path to scan (supports tokens).
- `SearchPattern` : File filter (e.g., `.` , `.jpg;.png`).
- `IncludeSubdirectories` : `true` for recursive deep scanning.

2. `AdvancedRenamerNode` **(Advanced Multi-Method Renamer)**

- **Purpose:** Batch renames files using an extensible pipeline of **9 sequential method steps**, emulating professional renaming suites.
- **Ports:**
- *Inputs:* `In` (`FileItemContext`)
- *Outputs:* `Out` (Success), `Skipped` (Collision skipped), `Error` (I/O or validation failure)
- **Parameters & Methods:**
- `RenameMode` : `Virtual` (default: projects name into context without altering source disk file) or `DirectInPlace` .
- `CollisionStrategy` : `AutoIncrement` (`_1` , `_2`), `Overwrite` , `Skip` , `Fail` .
- `MethodSteps` : Sequential JSON pipeline of cumulative transformation methods:

1. **New Name / Template:** Complete or partial template replacement with tags (or `{Tag}`).

2. **Search & Replace:** Substring or Regex pattern matching with capture groups (`$1` , `$2`), case sensitivity, and global replace.

3. **Insert Text:** Inserts strings at absolute or relative character positions.

4. **Delete Characters:** Removes character ranges by count or offset.

5. **Case Conversion:** `Lowercase` , `Uppercase` , `TitleCase` , `SentenceCase` , `CapitalizeFirst` .

6. **Incremental Numbering:** Sequential sequence generator with initial value, step, zero-padding (e.g., `001`), and reset trigger (*DirectoryChange*, *MetadataChange*, *Never*).

7. **Replace List (Substitutions):** Key-value mapping table for bulk dictionary replacements.

8. **Clean, Trim & Unicode Normalization:** Whitespace trimming, double space collapse, Windows illegal character sanitization (`\ / : * ? " < > |`), and Unicode normalization (`NFC` , `NFD` , `NFKC` , `NFKD`).

9. **Normalize & Pad Numbers:** Pads embedded numbers (e.g., `1 - track.mp3` $\rightarrow$ `01 - track.mp3` , `S1E2` $\rightarrow$ `S01E02` , `Ep. 3` $\rightarrow$ `Ep. 03`).

- **Regex Studio Assistant:** Interactive tester with live capture group inspection, replacement preview, and built-in preset library.
- **Visual Studio Window:** Live preview table with 100 loaded sample items and built-in presets (Digital Photo EXIF, ID3 Music, SEO Web, Corporate Docs).

3. `FileRelocatorNode` (File Relocator & Copier)

- **Purpose:** Copies or moves files to dynamically computed destinations, with optional SHA-256 integrity verification.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`
- **Parameters:**
- `Operation`: `Copy` (default, safe non-destructive) or `Move`.
- `DestinationDirectory`: Target directory template (e.g., `{DestinationRoot}\{Year}\{Month}`).
- `VerifyIntegrity`: `true` to verify source and target SHA-256 match.
- `CreateDirectories`: `true` to auto-create missing folders.

4. `OriginalFileActionNode` (Source File Lifecycle Action)

- **Purpose:** Centralized node for managing entry source files after downstream processing.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`
- **Parameters:**
- `ActionType`: `Keep` (default), `MoveToRecycleBin` (Windows Recycle Bin), `MoveToQuarantine`, `PermanentDelete`.
- `QuarantineDirectory`: Target path when action is quarantine.

5. `SafeRecycleDeleteNode` (Safe Delete to Recycle Bin)

- **Purpose:** Sends files to the Windows Recycle Bin using native Shell API.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Deleted`, `Error`

6. `EmptyDirectoryCleanerNode` (Empty Directory Cleaner)

- **Purpose:** Recursively scans and cleans folders that were left empty after moving operations.
- **Ports:**
- *Inputs:* `TriggerIn`
- *Outputs:* `Out`, `Error`
- **Parameters:**
- `TargetDirectory`: Root folder to clean.
- `Recursive`: `true` for deep scan.
- `IgnoreHiddenSystemFiles`: Ignores `Thumbs.db` and `.DS_Store`.

7. `OperationReportNode` (Interactive Operations Report)

- **Purpose:** Generates visual reports tracing complete file lifecycle, grouped by source folders.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Report`, `Error`
- **Parameters:**
- `ReportFormat`: `HTML` (interactive accordion), `Markdown`, `Text`, `JSON`, `CSV`.
- `ReportScope`: `Consolidated`, `PerFile`, `Both`.
- `GroupBy`: `Directory`, `Flat`, `Extension`, `Status`.
- `DestinationFolder`: Target folder for the report.
- `AutoOpenReport`: `true` to open in browser upon completion.

Category 2: Logic (Flow Control)

1. `SwitchCaseNode` (Dynamic Conditional Router)

- **Purpose:** Evaluates expressions or extensions and routes files to custom configured ports or `Default`.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* Dynamic case ports (e.g., `Images`, `Videos`, `Docs`) + `Default`.

2. `ExpressionFilterNode` (Boolean Condition Filter)

- **Purpose:** Evaluates numeric or string conditions (`SizeMB`, `Ext`, `CreationDate`) and routes to `True` or `False`.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `True`, `False`
- **Parameters:**
- `Property`: Property name (`SizeMB`, `Ext`, etc.).
- `Operator`: `>`, `<`, `≥`, `≤`, `==`, `≠`, `Contains`.
- `ComparisonValue`: Target comparison value.

3. `BatchBufferNode` (Batch Accumulator)

- **Purpose:** Buffers items until reaching N files or a maximum byte size before emitting the batch.
- **Ports:**
- *Inputs:* `ItemIn`, `ForceFlush`
- *Outputs:* `ItemOut`, `BatchCompleted`

4. `ForkJoinBarrierNode` **(Fork/Join Synchronization Barrier)**

- **Purpose:** Splices processing into multiple parallel branches and waits for all to finish before emitting `AllCompleted`.
- **Ports:**
- *Inputs:* `In`, `Branch1_Done`, `Branch2_Done`
- *Outputs:* `Fork1`, `Fork2`, `AllCompleted`

5. `ThrottleDelayNode` **(Rate Limiter & Delay)**

- **Purpose:** Introduces a configurable millisecond delay between items to prevent disk or API rate exhaustion.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`

Category 3: Hashing (Integrity & Deduplication)

1. `HashCalculatorNode` **(Cryptographic Checksum Calculator)**

- **Purpose:** Computes file hash and registers it in `Metadata["Hash:"]` *and* `{Hash:}`.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`
- **Parameters:**
- `Algorithm`: `SHA256`, `MD5`, `SHA512`, `SHA1`.
- `StoreInMetadataKey`: Key name (e.g., `Hash:SHA256`).

2. `DeduplicationFilterNode` **(Hash Deduplication Filter)**

- **Purpose:** Compares hashes within the current batch and splits unique items from duplicates.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Unique`, `Duplicate`, `Error`

Category 4: Archives (Compression & Extraction)

1. `SmartUnpackNode` **(Smart Archive Extractor)**

- **Purpose:** Unpacks ZIP, RAR, 7Z, and TAR archives with built-in multi-password dictionary support and Zip-Slip protection.

- **Ports:**
- *Inputs:* `In`
- *Outputs:* `ExtractedFile`, `Done`, `Error`
- **Parameters:**
- `DestinationPath`: Extraction folder.
- `FlattenHierarchy`: `true` to extract all files in a single flat directory.

2. `ArchiveFilterNode` **(Archive Format Filter)**

- **Purpose:** Filters valid archive containers from regular files.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `ArchiveOut`, `NonArchiveOut`

Category 5: Images & Media (Multimedia & EXIF)

1. `ImageOptimizerNode` **(Image Optimizer & Scaler)**

- **Purpose:** Resizes, compresses, and converts images to modern formats (**WebP**, **JPEG**, **PNG**) preserving aspect ratio.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`
- **Parameters:**
- `Width`: Target width in pixels, percentage, or auto (`""`).
- `Height`: Target height (default: `"100%"`).
- `TargetFormat`: `WebP`, `JPEG`, `PNG`.
- `Quality`: Visual compression quality (1-100, default: `80`).
- `OnlyDownscale`: `true` to prevent upscaling smaller images.

2. `ExifMetadataNode` **(EXIF Metadata Extractor)**

- **Purpose:** Extracts digital photo metadata (capture date, camera make/model, ISO, GPS coordinates) into `{Exif:*}` and `{Date:Taken}`.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`

3. `MediaTranscoderNode` **(FFmpeg Multimedia Converter)**

- **Purpose:** Transcodes video and audio streams using FFmpeg presets (H.264, HEVC/H.265, MP3, AAC).

- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`

Category 6: Scripting (Custom C# & JavaScript)

1. `CustomScriptNode` **(Dynamic Dual-Engine Scripting)**

- **Purpose:** Executes user scripts in **C# (Roslyn JIT with SHA256 caching)** or **JavaScript (Jint Sandboxed runtime)** with dynamic configurable ports and full metadata access.
- **Ports:** Configurable dynamically in Script Studio (`In`, `Out`, plus any user-defined inputs/outputs).
- **Features:**
- `Item` / `file`: Direct typed access to file attributes, metadata, and tags.
- `EmitAsync(port)` / `emit(port, item)`: Multi-port conditional dispatch.
- `Resolve(template)` / `resolve(template)`: Token resolver function.
- Interactive **Script Studio Window** with syntax highlighting, live testing, and script template library.

Category 7: Integrations (CLI & Webhooks)

1. `CliExecutionNode` **(External CLI Command Runner)**

- **Purpose:** Spawns external CLI processes (FFmpeg, Python, PowerShell) with tokenized arguments.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Success`, `Failed`
- **Parameters:**
- `ExecutablePath`: Path to executable.
- `ArgumentsTemplate`: Dynamic command arguments.
- `TimeoutSeconds`: Timeout in seconds (default: `60`).
- `CaptureOutputToMetadata`: Captures stdout into `Metadata["Cli:StdOut"]`.

2. `WebhookNotificationNode` **(HTTP POST Webhook Notifier)**

- **Purpose:** Sends JSON payloads via HTTP POST to external services (Discord, Slack, n8n, Zapier).
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Failed`

---

6. Example Flows and Built-In Templates

Template 1: Automated Photo Sorting by Date & Camera

```
[FolderSourceNode]
       │ (FileOut)
       ▼
[ExifMetadataNode]
       │ (Out)
       ▼
[AdvancedRenamerNode (<Exif:Date:yyyyMMdd>_<Exif:CameraModel>_<Index:D3>.<Ext>)]
       │ (Out)
       ▼
[FileRelocatorNode (Destination: {DestinationRoot}\{Year}\{Month})]
```

Template 2: Duplicate Cleaner to Windows Recycle Bin

```
[FolderSourceNode]
       │ (FileOut)
       ▼
[HashCalculatorNode (SHA-256)]
       │ (Out)
       ▼
[DeduplicationFilterNode]
  ├── (Unique)    ──▶ [LogOutputNode (Unique File Retained)]
  └── (Duplicate) ──▶ [OriginalFileActionNode (Action: MoveToRecycleBin)]
```

Template 3: Media Transcoding & Webhook Alert

```
[FolderSourceNode (*.mkv;*.avi)]
       │ (FileOut)
       ▼
[MediaTranscoderNode (Preset: "Fast 720p H.264 (MP4)")]
       │ (Out)
       ▼
[WebhookNotificationNode (URL: Discord Webhook, Payload: "Video {FileName} ready!")]
```
